Driver Drowsiness Detection System

A Deep Learning-Based Safety Project Using CNN and MobileNetV2

1. Project Overview

The Driver Drowsiness Detection project is a computer vision and deep learning application built entirely in Google Colab using Python and TensorFlow/Keras. The primary objective of this project is to detect signs of driver fatigue in real time by analyzing facial features — specifically the state of the driver's eyes (open or closed) and mouth (yawning or not yawning). Drowsy driving is one of the leading causes of road accidents worldwide, and this system aims to address that danger by classifying driver behavior into three fatigue levels: Alert, Mild Fatigue, and Severe Fatigue. The project compares two deep learning approaches — a custom-built Convolutional Neural Network (CNN) and a pretrained MobileNetV2 model using transfer learning — to determine which delivers better accuracy for real-world deployment.

2. Dataset Description and Structure

The dataset used in this project contains a total of 2,900 labeled images distributed across four distinct classes: Closed (eyes closed), Open (eyes open), no_yawn (mouth not yawning), and yawn (mouth yawning). These four classes are critical because they represent the visual cues most strongly associated with driver drowsiness. The dataset was originally stored as a compressed ZIP archive in Google Drive and was extracted programmatically at the start of the notebook using Python's built-in zipfile module. After extraction, the project verified the folder structure by walking through the directory tree and displaying the class-wise image counts, confirming a balanced distribution across all four categories — an important factor for avoiding model bias during training.

Once the dataset was verified, sample images from each class were displayed in a 2x4 grid using matplotlib to visually confirm the variety and quality of training data. This exploratory step helped confirm that the images cover a range of lighting conditions, face orientations, and individual differences, which is essential for building a robust and generalizable model.

3. Dataset Splitting and Preprocessing

Since the original dataset came with only a training folder, a custom splitting strategy was implemented to divide the data into three subsets: 70% for training, 15% for validation, and 15% for testing. This was accomplished using Python's shutil and random modules — each class was independently shuffled and then split according to the defined ratios, with copies of the images placed into newly created split_dataset/train, split_dataset/valid, and split_dataset/test directories. This resulted in approximately

2,029 training images, 435 validation images, and 436 test images, maintaining a consistent class balance across all three splits.

Data preprocessing was handled using TensorFlow's ImageDataGenerator class. For the training data, augmentation techniques were applied to artificially increase variety and improve model generalization — this included random rotation (up to 20 degrees), zoom (up to 20%), horizontal flipping, and brightness variation (between 0.8x and 1.2x). All images were normalized by rescaling pixel values from the range 0–255 to 0–1. For the validation and test datasets, only normalization was applied without any augmentation, ensuring that evaluation reflects real-world performance. All images were resized to 224x224 pixels to match the input requirements of both the custom CNN and the MobileNetV2 model.

4. Custom CNN Model Architecture

The first model built in this project is a custom Convolutional Neural Network (CNN) designed from scratch using TensorFlow's Keras Sequential API. The architecture consists of four convolutional blocks, each containing a Conv2D layer (with increasing filter sizes of 32, 64, 128, and 256), a BatchNormalization layer to stabilize and speed up training, and a MaxPooling2D layer to reduce spatial dimensions. After the convolutional blocks, the output is flattened and passed through two fully connected Dense layers with 512 and 256 neurons respectively, each followed by Dropout layers (0.5 and 0.3) to reduce overfitting. The final output layer uses 4 neurons with a softmax activation function to produce probability scores across the four classes.

The model was compiled using the Adam optimizer with categorical cross-entropy as the loss function and accuracy as the evaluation metric. Training was conducted for up to 15 epochs with two callbacks: EarlyStopping (monitoring validation accuracy with a patience of 5 epochs to halt training if improvement stalls) and ModelCheckpoint (to save only the best version of the model based on validation accuracy). This approach ensured that the final saved model represents peak performance and not an overfitted later state. The CNN achieved a final test accuracy of 80.73%, performing particularly well on the Closed and Open eye classes (95% each) but struggling with yawn detection (52%).

5. MobileNetV2 Transfer Learning Model

The second and superior model in this project uses MobileNetV2, a lightweight and highly efficient pretrained convolutional neural network originally trained on the ImageNet dataset containing over one million images across 1,000 categories. By leveraging transfer learning, the model inherits MobileNetV2's rich understanding of visual features such as edges, textures, and shapes, allowing it to adapt quickly to the drowsiness detection task with far fewer training samples than would otherwise be

required.

The implementation loads MobileNetV2 without its top classification layer (include_top=False) and freezes all the base layers so their pretrained weights are preserved during initial training. Custom classification layers are then added on top: a GlobalAveragePooling2D layer to compress spatial features, followed by Dense layers of 256 and 128 neurons with ReLU activation, each paired with Dropout layers (0.3 and 0.2), and finally a 4-neuron softmax output layer. The same training callbacks — EarlyStopping and ModelCheckpoint — were used as in the CNN training. MobileNetV2 significantly outperformed the custom CNN, achieving a test accuracy of 92.20%, with perfect classification on both Closed (100%) and Open (100%) eye classes, and strong performance on no_yawn (86%) and yawn (84%).

6. Training Visualization and Performance Monitoring

After training both models, their learning progress was visualized through accuracy and loss curves plotted using matplotlib. Two side-by-side graphs were generated for each model — one showing training accuracy versus validation accuracy across epochs, and one showing training loss versus validation loss. These charts serve an important diagnostic purpose: they reveal whether the model is underfitting (both curves remain low), overfitting (training is high but validation is low), or generalizing well (both curves are high and close together). For MobileNetV2, the curves showed smooth convergence with strong validation accuracy, confirming that the transfer learning approach generalized well to the drowsiness detection task.

7. Model Evaluation and Confusion Matrix Analysis

Both models were evaluated on the held-out test dataset, and their results were analyzed using confusion matrices and detailed classification reports. A confusion matrix is a grid that shows how many images from each actual class were predicted as each possible class, making it easy to identify which specific categories a model confuses most. The CNN's confusion matrix (visualized using seaborn's heatmap with a blue color scheme) revealed notable misclassifications in the yawn category, while the MobileNetV2 matrix (green color scheme) showed far fewer errors overall. The classification report for each model provided per-class precision, recall, and F1-score, offering a granular view of model strengths and weaknesses beyond a single accuracy number.

The final comparison clearly established MobileNetV2 as the winning model with 92.20% test accuracy versus the CNN's 80.73%, a difference of nearly 12 percentage points — a substantial margin in machine learning terms. This outcome is expected given that MobileNetV2 brings millions of prelearned parameters, while the custom CNN had to learn all visual representations from scratch using only ~2,000 training images.

8. Decision Fusion — Three-Level Fatigue Classification

A key innovation in this project is the Decision Fusion layer that maps the four raw classification outputs into a meaningful, actionable three-level fatigue system. The mapping logic is as follows: if the predicted class is Open or no_yawn, the driver is classified as Level 0 — Alert; if the prediction is yawn, the driver is classified as Level 1 — Mild Fatigue; and if the prediction is Closed, the driver is classified as Level 2 — Severe Fatigue. This abstraction transforms a fine-grained visual classification into a practical safety assessment that can directly drive real-world interventions.

The decision fusion logic was applied to all test set predictions from MobileNetV2, producing a distribution of fatigue levels across the test frames: approximately 53.2% were classified as Alert, 21.6% as Mild Fatigue, and 25.2% as Severe Fatigue. This distribution was visualized with a color-coded bar chart — green for Alert, orange for Mild Fatigue, and red for Severe Fatigue — providing an intuitive summary of driver state across the entire test dataset.

9. Fatigue Progression Curve and Driving Session Simulation

To simulate real-world deployment, the project implemented a driving session analysis that groups predictions into one-minute intervals (assuming 10 frames per minute) and computes the average fatigue level per interval. This produces a continuous Fatigue Progression Curve over the simulated driving session — a line graph that plots how driver alertness changes over time. The y-axis represents the three fatigue levels (0 = Alert, 1 = Mild, 2 = Severe), and background colored bands mark the three zones: green for the Alert zone (below 0.5), orange for Mild Fatigue (0.5 to 1.5), and red for Severe Fatigue (above 1.5).

The simulation covered a total session of 43 minutes with an average fatigue level of 0.72, placing the overall session in the Mild Fatigue range. The code also identifies specific transition minutes — when the driver crosses from Alert to Mild Fatigue, and from Mild to Severe Fatigue — providing actionable timing information that could be used to trigger alerts at precisely the right moment in a real deployment scenario.

10. Business Recommendations and Real-World Application

The final section of the project translates the technical results into actionable safety recommendations mapped to each fatigue level. For Alert drivers (Level 0), no system intervention is required. For Mild Fatigue drivers (Level 1), the system recommends playing an audible warning sound to prompt the driver to take a break. For Severe Fatigue drivers (Level 2), the recommended response is an emergency alert combined with automatic braking — a critical life-saving intervention for the most dangerous drowsiness state.

These recommendations align the machine learning output directly with automotive safety engineering requirements, demonstrating how the model can be integrated into a real Advanced Driver Assistance System (ADAS). The project concludes with a bar chart comparing the final test accuracy of the Custom CNN (80.73%) against MobileNetV2 (92.20%), with the MobileNetV2 bar clearly standing taller — a fitting visual conclusion to a project that demonstrates the power of transfer learning for safety-critical computer vision applications.

11. Why MobileNetV2 Outperforms the Custom CNN

The reason MobileNetV2 significantly outperforms the custom CNN in this project comes down to the fundamental advantage of transfer learning. MobileNetV2 was pretrained on ImageNet — a massive dataset of over one million images — and has already learned to recognize a rich hierarchy of visual features: low-level features like edges and gradients in early layers, and high-level features like textures, shapes, and object parts in deeper layers. When fine-tuned on the drowsiness dataset, these learned representations provide an extremely strong starting point, requiring the model to learn only the task-specific mapping from these rich features to the four drowsiness classes.

In contrast, the custom CNN must learn every feature from scratch using only around 2,000 training images — a challenging task for a deep network. While the CNN still achieves a respectable 80.73% accuracy, it struggles particularly with subtle visual distinctions like yawning (52% accuracy), where the difference between yawn and no_yawn requires nuanced mouth-shape recognition that benefits from the richer feature representations learned by MobileNetV2 from a much larger and more diverse training set. Additionally, MobileNetV2's depthwise separable convolution architecture makes it computationally efficient, making it ideal not just for accuracy but also for potential deployment on embedded systems in vehicles.

Project Summary at a Glance

Total Dataset: 2,900 images across 4 classes (Closed, Open, no_yawn, yawn)

Data Split: 70% Train | 15% Validation | 15% Test

Model 1: Custom CNN — 4 Conv blocks + Dense layers → 80.73% accuracy

Model 2: MobileNetV2 Transfer Learning → 92.20% accuracy

Fatigue Levels: Level 0: Alert | Level 1: Mild Fatigue | Level 2: Severe Fatigue

Winner: MobileNetV2 (+11.47% over custom CNN)

Platform: Google Colab with TensorFlow/Keras

This project demonstrates end-to-end machine learning pipeline design — from data preparation and model building to evaluation, decision logic, and real-world recommendation — making it a strong example of applied deep learning for automotive safety.